

Foundations of 4D/6D Object Capture for Virtual Environments

Differentiable Rendering



March 4, 2026



→ **Colors and Rendering**





The background of the slide features a virtual environment. On the left, a person wearing a grey and white hoodie and light blue pants stands with one hand on their hip and the other raised. On the right, a white rabbit with a red collar sits. The background is a repeating pattern of white star-like shapes on a dark blue background.

Today: Differentiable Rendering

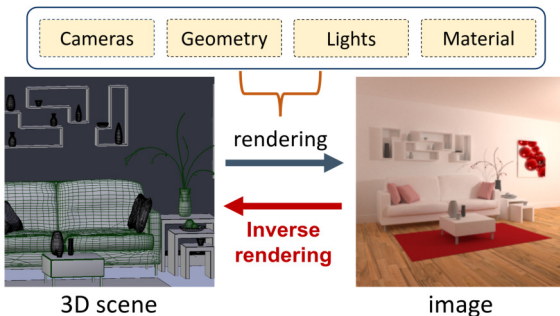
Since we now also know the fundamentals of rendering: Is rendering somehow related to reconstruction?

Forward Rendering

- ▶ Given: Cameras, Geometry, Lights, Material
- ▶ Goal: Simulated image

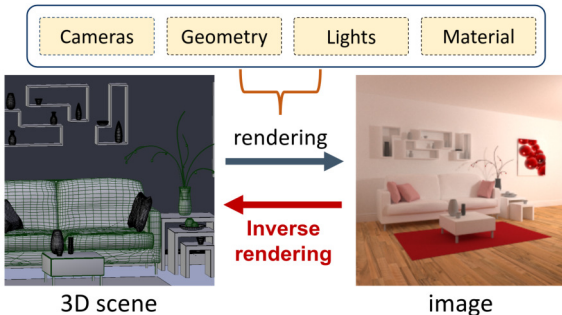
Inverse Rendering

- ▶ Given: Set of measurements (images) of the scene
- ▶ Goal: (Subset of) Cameras, Geometry, Lights, Material



Task: More formally, the task is to find scene parameters Θ^* that best reproduce the given images I_i :

$$\Theta^* = \arg \min_{\Theta} \sum_{i=1}^m \left\| \underbrace{R}_{\text{render function}} \left(\underbrace{\Theta}_{\text{scene parameters}} \right) - I_i \right\|^2$$



→ Optimization

To perform optimization, an objective function of the parameters $\Theta \in \mathbb{R}^n$, typically called *Loss function*¹ or Error function, is defined:

$$\mathcal{L}(\Theta) = \sum_{i=1}^m \rho(r_i(\Theta))$$

► ρ : Penalty function, e.g.:

Linear (L_1)

$$\rho(x) = |x|$$

Quadratic (L_2)

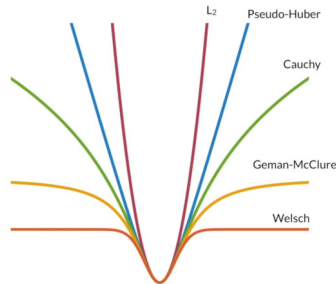
$$\rho(x) = x^2$$

Welsch

$$\rho(x) = 1 - e^{-\frac{1}{2}x^2}$$

► r_i : Residual between current state Θ and target, e.g.:

$$r_i(\Theta) = \|R(\Theta) - I_i\|$$



¹J. T. Barron. "A General and Adaptive Robust Loss Function". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019.

Linear Least Squares:

$$\mathcal{L}(\Theta) = \sum_{i=1}^m r_i(\Theta)^2$$

- ▶ $\rho(x) = x^2$
- ▶ r_i is a linear function of Θ

We can rewrite this equation in matrix form:

$$\mathcal{L}(\Theta) = \|A\Theta - b\|^2 \quad A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$$

Solution: Use normal equations

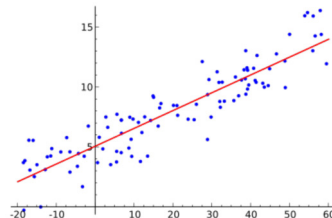
$$A^T A \Theta = A^T b \quad \Rightarrow \quad \Theta^* = (A^T A)^{-1} A^T b$$

→ For this to work, we need $m \geq n$ different inputs

Example: Line fitting

$$y = m \cdot x + t \quad \Rightarrow \quad r_i(m, t) = y_i - m \cdot x_i - t$$

$$x_i = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} \quad y_i = \begin{pmatrix} 6 \\ 5 \\ 7 \\ 10 \end{pmatrix}$$



The solution for the parameters m and t is then:

$$\mathcal{L}(m, t) = \left\| \begin{pmatrix} 6 \\ 5 \\ 7 \\ 10 \end{pmatrix} - m \cdot \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix} - t \right\|^2 = \left\| \overbrace{\begin{pmatrix} 6 \\ 5 \\ 7 \\ 10 \end{pmatrix}}^{\mathbf{b}} - \overbrace{\begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \\ 4 & 1 \end{pmatrix}}^{\mathbf{A}} \cdot \begin{pmatrix} m \\ t \end{pmatrix} \right\|^2$$
$$\begin{pmatrix} m \\ t \end{pmatrix} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b} = \begin{pmatrix} 30 & 10 \\ 10 & 4 \end{pmatrix}^{-1} \begin{pmatrix} 77 \\ 28 \end{pmatrix} = \begin{pmatrix} 1.4 \\ 3.5 \end{pmatrix}$$

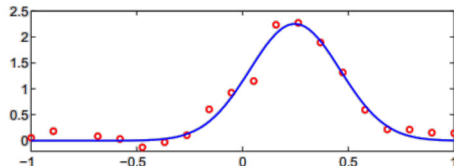
Non-Linear Least Squares:

$$\mathcal{L}(\Theta) = \sum_{i=1}^m r_i(\Theta)^2$$

- ▶ $\rho(x) = x^2$
- ▶ r_i is not a linear function of Θ

Example: Gaussian fitting

$$y = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \Rightarrow r_i(\mu, \sigma) = y_i - \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}$$



Problem: No closed-form analytic solution available in general!

Problem: No closed-form analytic solution available in general!

Solution: Approximate Θ^* and use numerical method:

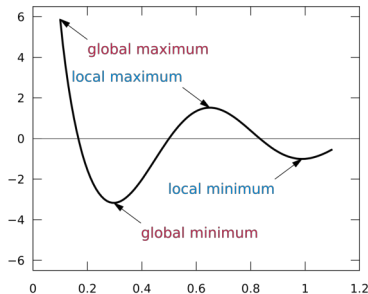
$$\Theta_{k+1} = \Theta_k + \Delta\Theta_k$$

where Θ_0 is set to some initial value

How to choose $\Delta\Theta_k$?

- ▶ No perfect approach for this!
- ▶ Only *local minimum* can be guaranteed
- ▶ Can still find good local minimum that is not too far from global minimum

→ "Simplest Approach": Gradient Descent



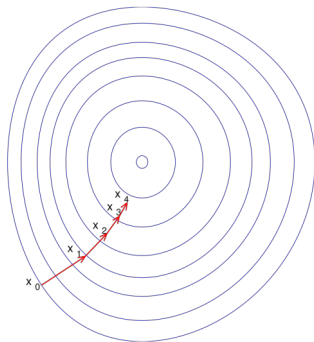
Idea: Follow the negative gradients to find a local minimum:

$$\Theta_{k+1} = \Theta_k - t_k \cdot \nabla f(\Theta_k)$$

- Need to compute partial derivatives

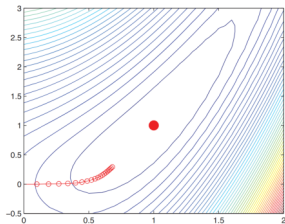
$$\nabla f(\Theta) = \left(\frac{\partial f}{\partial \theta_1}(\Theta), \dots, \frac{\partial f}{\partial \theta_n}(\Theta) \right)^T \in \mathbb{R}^n$$

- Need to determine step size t_k

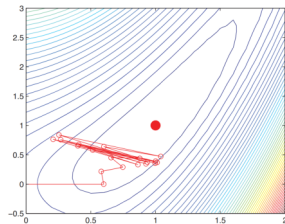


Question: How to estimate suitable step sizes t_k ?

Too small $\rightarrow$ Slow Convergence



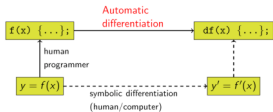
Too large $\rightarrow$ May diverge to undesired result



Popular choices for t_k :

- Constant
- With line search
- With momentum

Numerical Differentiation



Based on definition of derivative:

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h \mathbf{e}_i) - f(\mathbf{x})}{h}$$

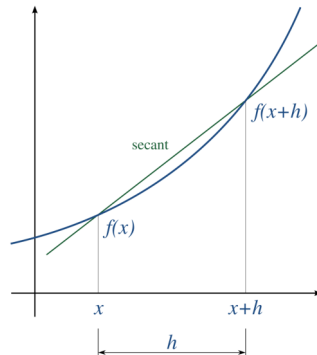
- Approximation directly from definition:

Forward Difference $\frac{\partial f}{\partial x_i}(\mathbf{x}) \approx \frac{f(\mathbf{x} + h \mathbf{e}_i) - f(\mathbf{x})}{h}$

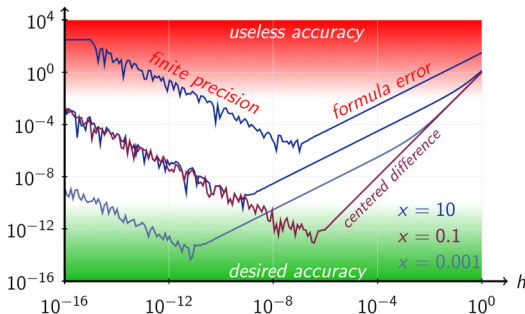
- More accurate alternative:

Central Difference $\frac{\partial f}{\partial x_i}(\mathbf{x}) \approx \frac{f(\mathbf{x} + h \mathbf{e}_i) - f(\mathbf{x} - h \mathbf{e}_i)}{2h}$

→ Error: $\mathcal{O}(h^2)$ instead of $\mathcal{O}(h)$



Problem: Step size h needs to be carefully tuned to achieve acceptable results



Solution:

1. Higher-order finite differences, e.g. 3rd or 4th order → More expensive, less scalable
2. Different differentiation methods → More scalable

Based on "manual" way of computing derivatives → Accurate

Downsides:

- ▶ Operates and (thus) requires mathematical expressions of function $f(x)$
- ▶ Resulting expression of derivative can become very verbose

```
daly@spiff: ~  
daly@spiff:~$ export AXIOM=/space/lambda1/mnt/ubuntu  
daly@spiff:~$ export PATH=$AXIOM/bin:$PATH  
daly@spiff:~$ axiom -nox  
AXIOM Computer Algebra System  
Version: Axiom (July 2009)  
Timestamp: Sunday August 9, 2009 at 15:26:54  
-----  
Issue )copyright to view copyright notices.  
Issue )summary for a summary of useful system commands.  
Issue )quit to leave AXIOM and return to shell.  
-----  
Re-reading compress.daase Re-reading interp.daase  
Re-reading operation.daase  
Re-reading category.daase  
Re-reading browse.daase  
(1) ->  
(1) -> f:=x/sqrt(x^4+10*x^2-96*x-71)  

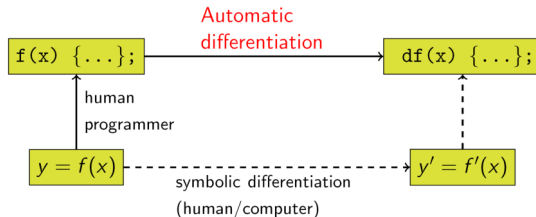
$$(1) \quad \frac{x}{\sqrt{x^4 + 10x^2 - 96x - 71}}$$
  
Type: Expression Integer  
(2) -> integrate(f,x)  
(2)  
log  

$$\frac{\log\left(\frac{(x^6 + 15x^4 - 80x^3 + 27x^2 - 528x + 781)\sqrt{x^4 + 10x^2 - 96x - 71} - x^8}{-20x^6 + 128x^5 - 54x^4 + 1408x^3 - 3124x^2 - 10001}\right)}{8}$$
  
Type: Union(Expression Integer,...)  
(3) -> []
```

Similar to symbolic differentiation by computing **analytic derivatives** → Accurate

Idea:

- ▶ Compute the value of the derivative of f at some given point $x_0 \dots$
- ▶ $\dots$ like an implementation of f would compute the corresponding result for x_0



→ If we have an implementation of f , we can "auto-generate" an implementation of its derivative $\frac{\partial f}{\partial x}(x_0)$

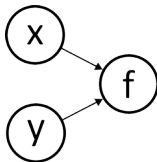
To compute the derivatives, we can break down a function into smaller functions:



In 1D for $g(f(x))$, this boils down to the chain rule:

$$\frac{\partial}{\partial x} g(f(x)) = \frac{\partial g}{\partial f} \frac{\partial f}{\partial x}$$

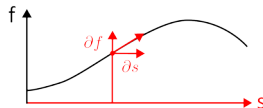
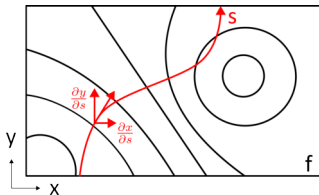
Question: What if a function depends on several inputs?



→ Consider the partial derivatives $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$

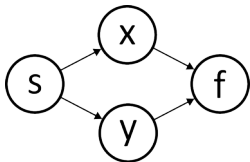
Let us now assume that we are moving along a path:

$$f(x(s), y(s))$$



To calculate the partial derivative of f with respect to s , we must consider

1. the derivative of f with respect to x and y
2. the influence of s on x and y

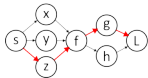
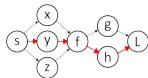
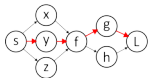
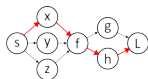
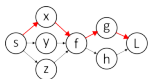
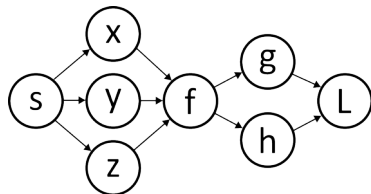


$$\frac{\partial f}{\partial s} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial s}$$

→ In the compute graph, we must add up both paths on which s influences f

Problem: Complexity of derivative can grow very fast

- ▶ Simplest case: Single path $\rightarrow$ Linear complexity
- ▶ Worst case: Multiple variables per depth of the graph $\rightarrow$ **Exponential complexity**

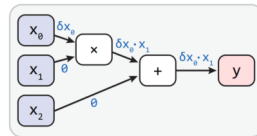


$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial s} = & \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial f} \frac{\partial f}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial f} \frac{\partial f}{\partial x} \frac{\partial x}{\partial s} \\ & + \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial f} \frac{\partial f}{\partial y} \frac{\partial y}{\partial s} + \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial f} \frac{\partial f}{\partial y} \frac{\partial y}{\partial s} \\ & + \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial s} + \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial s}\end{aligned}$$

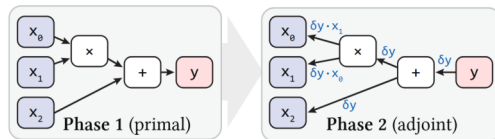
Problem: Complexity of derivative can grow very fast

Use dynamic programming to avoid duplicate computations

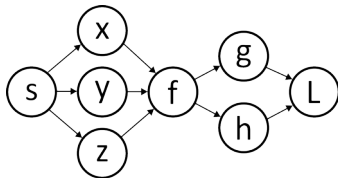
- **Forward Mode:** Start at input and gradually compute partial derivatives w.r.t. input until reaching output



- **Reverse Mode:**
 1. Evaluate calculation graph ("forward-pass") and save intermediate results
 2. Start at output ("backward-pass") and gradually compute partial derivatives of output w.r.t. individual nodes until reaching input



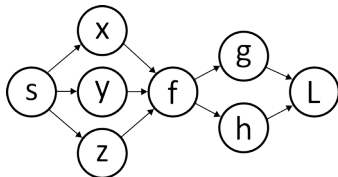
Forward Mode:



$$\begin{aligned} \frac{\partial x}{\partial s} & \Rightarrow \frac{\partial f}{\partial s} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial f}{\partial y} \frac{\partial y}{\partial s} + \frac{\partial f}{\partial z} \frac{\partial z}{\partial s} \Rightarrow \begin{aligned} \frac{\partial g}{\partial s} &= \frac{\partial g}{\partial f} \frac{\partial f}{\partial s} \\ \frac{\partial h}{\partial s} &= \frac{\partial h}{\partial f} \frac{\partial f}{\partial s} \end{aligned} \Rightarrow \boxed{\frac{\partial \mathcal{L}}{\partial s}} = \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial s} + \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial s} \end{aligned}$$

- ▶ Intermediate derivatives $\frac{\partial \dots}{\partial s}$ have dimension $\dots \times |s|$ → Preferred if $|\mathcal{L}| \gg |s|$
- ▶ Intermediate derivatives w.r.t. s cannot be used for optimization w.r.t. $\mathcal{L}$ → Need derivatives of $\mathcal{L}$!

Reverse Mode:



$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial g} &\Rightarrow \frac{\partial \mathcal{L}}{\partial f} = \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial f} + \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial f} \Rightarrow \frac{\partial \mathcal{L}}{\partial y} = \frac{\partial \mathcal{L}}{\partial f} \frac{\partial f}{\partial y} \Rightarrow \boxed{\frac{\partial \mathcal{L}}{\partial s}} = \frac{\partial \mathcal{L}}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial s} + \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial s} \\
 \frac{\partial \mathcal{L}}{\partial h} &\Rightarrow \frac{\partial \mathcal{L}}{\partial f} = \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial f} + \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial f} \Rightarrow \frac{\partial \mathcal{L}}{\partial y} = \frac{\partial \mathcal{L}}{\partial f} \frac{\partial f}{\partial y} \Rightarrow \boxed{\frac{\partial \mathcal{L}}{\partial s}} = \frac{\partial \mathcal{L}}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial s} + \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial s} \\
 \frac{\partial \mathcal{L}}{\partial x} &\Rightarrow \frac{\partial \mathcal{L}}{\partial f} = \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial f} + \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial f} \Rightarrow \frac{\partial \mathcal{L}}{\partial y} = \frac{\partial \mathcal{L}}{\partial f} \frac{\partial f}{\partial y} \Rightarrow \boxed{\frac{\partial \mathcal{L}}{\partial s}} = \frac{\partial \mathcal{L}}{\partial x} \frac{\partial x}{\partial s} + \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial s} + \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial s}
 \end{aligned}$$

- ▶ Intermediate derivatives $\frac{\partial \mathcal{L}}{\partial \dots}$ have dimension $|\mathcal{L}| \times \dots \rightarrow$ Preferred if $|\mathcal{L}| \ll |s|$
- ▶ Intermediate derivatives of $\mathcal{L}$ can be directly used to optimize the parameters of the graph

Summary:

► Forward Mode:

$$\frac{\partial \dots}{\partial s} \text{ with dimension } \dots \times |s|$$

► Reverse Mode:

$$\frac{\partial \mathcal{L}}{\partial \dots} \text{ with dimension } |\mathcal{L}| \times \dots$$

→ Almost always use Reverse Mode since $|\mathcal{L}| = 1$, especially in Deep Learning!

```
import torch

def identity(x):
    return x

x = torch.randn(10,1,requires_grad=True) # "Ground Truth data x"
y = torch.randn(10,1,requires_grad=False) # "Ground Truth data y"
W = [torch.randn(10,10,requires_grad=True) for i in range(3)] # Weight-Matrices
b = [torch.randn(10,1,requires_grad=True) for i in range(3)] # Bias-Vectors
f = [torch.sigmoid,torch.sigmoid,identity] # Activation functions
a = [] # activations
z = [x] # outputs of individual Layers

# loop over network layers
for k in range(3):
    a.append(torch.matmul(W[k],z[-1])+b[k]) # compute activations
    z.append(f[k](a[-1])) # apply activation function

# compute square error loss between network prediction and ground truth y
Loss = torch.sum((z[-1]-y)**2)

# compute Gradients with backpropagation algorithm
Loss.backward()

# print out Gradients
print(f"x.grad = {x.grad.data}")

for k in range(3):
    print(f"Layer {k}:")
    print(f"W[{k}].grad = {W[k].grad.data}")
    print(f"b[{k}].grad = {b[k].grad.data}")
```

Task: Given some images I_i , find scene parameters Θ^* that best reproduce them

$$\Theta^* = \arg \min_{\Theta} \sum_{i=1}^m \left\| \underbrace{R}_{\text{render function}} \left(\underbrace{\Theta}_{\text{scene parameters}} \right) - I_i \right\|^2$$

Using Gradient Descent optimization leads to:

$$\begin{aligned} \Theta_{k+1} &= \Theta_k - t_k \cdot \nabla \left(\sum_{i=1}^m \|R(\Theta_k) - I_i\|^2 \right) \\ &= \Theta_k - t_k \cdot \frac{\partial}{\partial \Theta} \left(\sum_{i=1}^m \|R(\Theta_k) - I_i\|^2 \right) \\ &= \Theta_k - t_k \cdot \sum_{i=1}^m 2 \cdot (R(\Theta_k) - I_i) \cdot \frac{\partial}{\partial \Theta} R(\Theta_k) \end{aligned}$$

Question: How to get $\frac{\partial R}{\partial \Theta}$?

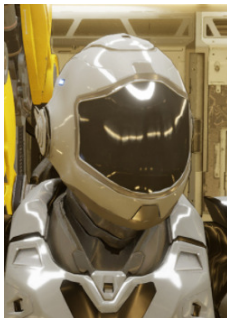


Question: How to get $\frac{\partial \mathbf{R}}{\partial \Theta}$?

→ Depends on the rendering technique



$\frac{\partial \text{Rasterization}}{\partial \Theta}$



$\frac{\partial \text{Ray Tracing}}{\partial \Theta}$

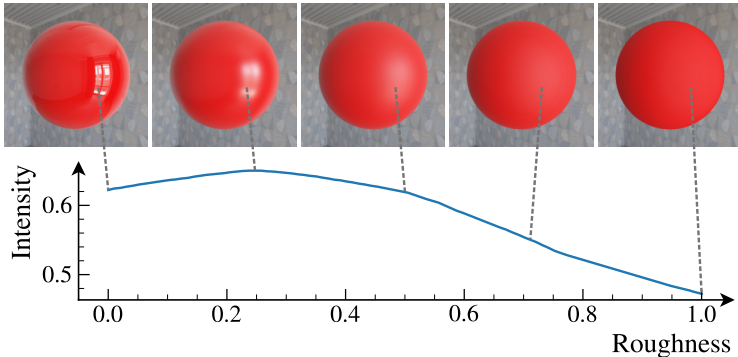


$\frac{\partial \text{Path Tracing}}{\partial \Theta}$

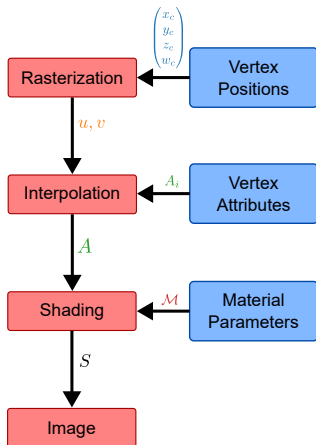
For rasterization, the rendering equation can be simplified to

$$R(\Theta) = L_o(x, l | \Theta) = L_e(x, v | \Theta) + \sum_{l_s} f_{\text{BRDF}}(x, l_s, v | \Theta) L_{e, l_s}(x, l_s | \Theta) \langle n | l_s \rangle$$

Example: How does $R(\Theta)$ change if we change for instance the roughness parameter?



Reminder: Deferred Shading

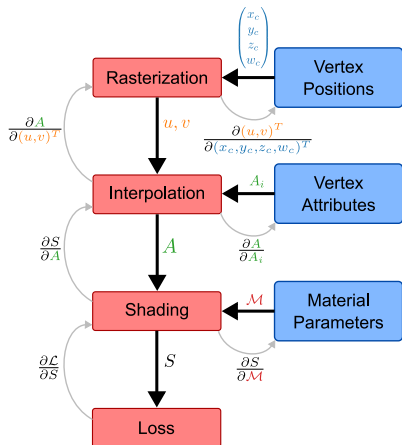


In general, the **forward pass** of a deferred rasterizer takes the parameters

$$\Theta = \left((x_c, y_c, z_c, w_c)^T, A_i, \mathcal{M} \right)^T$$

- ▶ Clip-space vertex positions $(x_c, y_c, z_c, w_c)^T$
- ▶ Vertex attributes A_i like normals, uv-parameterization, etc.
- ▶ Material Parameters $\mathcal{M}$
 - ▶ Base color: κ_{base}
 - ▶ Roughness: roughness
 - ▶ Metalness: metallic

Reminder: Deferred Shading



Task: Compute the derivative with respect to the input parameters Θ

- Clip-space vertex positions

$$\frac{\partial \mathcal{L}}{\partial (x_c, y_c, z_c, w_c)^T} = \frac{\partial \mathcal{L}}{\partial S} \frac{\partial S}{\partial A} \frac{\partial A}{\partial (u, v)^T} \frac{\partial (u, v)^T}{\partial (x_c, y_c, z_c, w_c)^T}$$

- Vertex attributes

$$\frac{\partial \mathcal{L}}{\partial A_i} = \frac{\partial \mathcal{L}}{\partial S} \frac{\partial S}{\partial A} \frac{\partial A}{\partial A_i}$$

- Material Parameters

$$\frac{\partial \mathcal{L}}{\partial \mathcal{M}} = \frac{\partial \mathcal{L}}{\partial S} \frac{\partial S}{\partial \mathcal{M}}$$

Forward Pass

1. Take in clip-space vertex positions $(x_c, y_c, z_c, w_c)^T$ and map to image space $\mathbf{p} \in \mathbb{R}^2$ via perspective division
2. Each pixel stores which triangle it belongs to and the corresponding barycentric coordinates $(u, v)^T$:

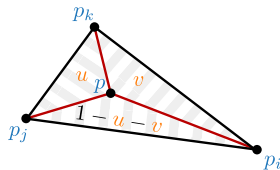
$$\begin{aligned}\mathbf{p} &= u \mathbf{p}_i + v \mathbf{p}_j + (1 - u - v) \mathbf{p}_k \\ \mathbf{p} - \mathbf{p}_k &= (\mathbf{p}_i - \mathbf{p}_k) u + (\mathbf{p}_j - \mathbf{p}_k) v \\ (u, v)^T &= \begin{pmatrix} \mathbf{p}_i - \mathbf{p}_k & \mathbf{p}_j - \mathbf{p}_k \end{pmatrix}^{-1} (\mathbf{p} - \mathbf{p}_k)\end{aligned}$$

Backward Pass

- Need to compute partial derivative:

$$\frac{\partial (u, v)^T}{\partial \mathbf{p}} = \begin{pmatrix} \mathbf{p}_i - \mathbf{p}_k & \mathbf{p}_j - \mathbf{p}_k \end{pmatrix}^{-1} \in \mathbb{R}^{2 \times 2}$$

- Perspective division is easily differentiated: $\frac{\partial \mathbf{p}}{\partial (x_c, y_c, z_c, w_c)^T}$



Forward Pass

1. Given: Per-vertex parameters A_i like normals, texture coordinates, etc.
2. Compute interpolated attribute within a triangle with vertex indices i, j, k via barycentric coordinates:

$$A = u A_i + v A_j + (1 - u - v) A_k$$

Backward Pass

- For propagating to the vertex positions:

$$\frac{\partial A}{\partial (u, v)^T} = \begin{pmatrix} A_i - A_k & A_j - A_k \end{pmatrix}$$

- For propagating to the vertex attributes:

$$\frac{\partial A}{\partial A_l} = \begin{cases} u & \text{if } l = i \\ v & \text{if } l = j \\ (1 - u - v) & \text{if } l = k \end{cases}$$

Reminder:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial (x_c, y_c, z_c, w_c)^T} &= \frac{\partial \mathcal{L}}{\partial S} \frac{\partial S}{\partial A} \frac{\partial A}{\partial (u, v)^T} \frac{\partial (u, v)^T}{\partial (x_c, y_c, z_c, w_c)^T} \\ \frac{\partial \mathcal{L}}{\partial A_i} &= \frac{\partial \mathcal{L}}{\partial S} \frac{\partial S}{\partial A} \frac{\partial A}{\partial A_i} \end{aligned}$$

Shading

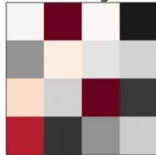
- Takes in vertex attributes $\mathcal{A}$ and material parameters $\mathcal{M}$
→ Use automatic differentiation to propagate the gradients

$$\frac{\partial S}{\partial \mathcal{A}} \quad , \quad \frac{\partial S}{\partial \mathcal{M}}$$

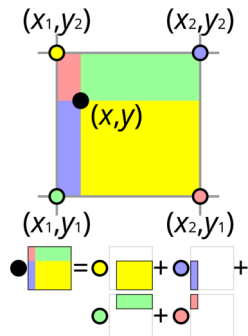
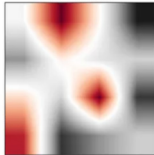
Texture Mapping

- Nearest neighbor lookup: not differentiable
→ Bilinear interpolation

nearest neighbour

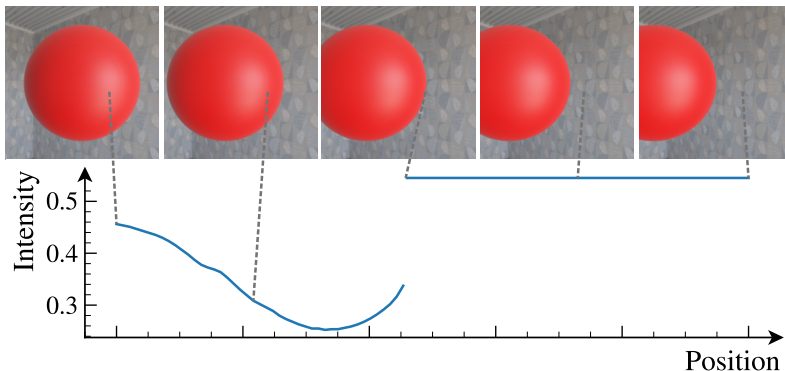


bilinear



If Θ only consists of material parameters (and f_{BRDF} is continuous), then $R(\Theta)$ is differentiable

Question: What happens if we optimize the position or geometry of objects in the scene?

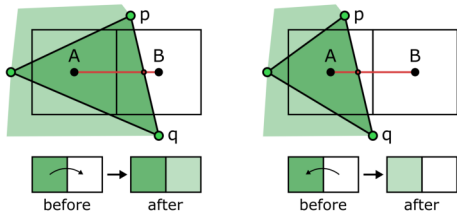


Problem: Rendering function becomes discontinuous on visibility edges!

How to handle discontinuities on visibility edges? $\rightarrow$ nvdiffrast²

Forward Pass

1. Find neighboring pixels with mismatching triangle IDs
2. Look for silhouette edges in triangles (edges with only one neighboring triangle after projection)
3. Compute blend weights according to triangle's pixel coverage

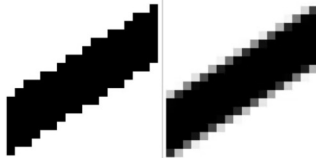


Backward Pass

- For each antialiased pixel pair: Compute gradient of blending weights w.r.t. vertex positions and update gradients accordingly

Aliased

Anti-Aliased



²S. Laine et al. “Modular Primitives for High-Performance Differentiable Rendering”. In: *ACM Transactions on Graphics* (2020).

In case of Path Tracing (and Ray Tracing), we need to consider multiple bounces and to differentiate through the integral:

$$R(\Theta) = L_o(x, l | \Theta) = L_e(x, v | \Theta) + \int_{\Omega} f_{\text{BRDF}}(x, l, v | \Theta) L_i(x, l | \Theta) \langle n | l \rangle dl$$

Use [Reynolds transport theorem](#) to compute derivatives:

$$\begin{aligned} \frac{\partial}{\partial \theta_i} \left(\int_{\mathcal{R}(\theta_i)} f(x, \theta_i) dV \right) &= \int_{\mathcal{R}(\theta_i)} \frac{\partial}{\partial \theta_i} f(x, \theta_i) dV && \text{[Interior Integral]} \\ &+ \int_{\partial \mathcal{R}(\theta_i)} \underbrace{g(x, \theta_i)}_{\substack{\text{simplified notation} \\ \rightarrow \text{depends on } f}} dA && \text{[Boundary Integral]} \end{aligned}$$

→ 3D generalization of **Leibniz integral rule** on parameter-dependent region $\mathcal{R}(\theta_i)$

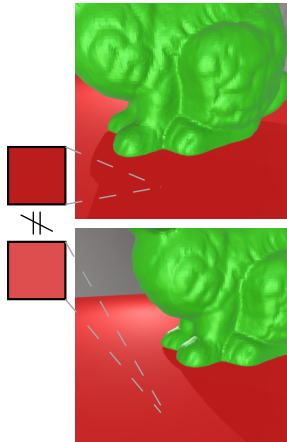
Question: Can we use Antialiasing again to resolve this boundary integral?

Question: Can we use Antialiasing again to resolve this boundary integral?

Unfortunately, no! → Boundary region $\partial\mathcal{R}(\theta_i)$ depends on

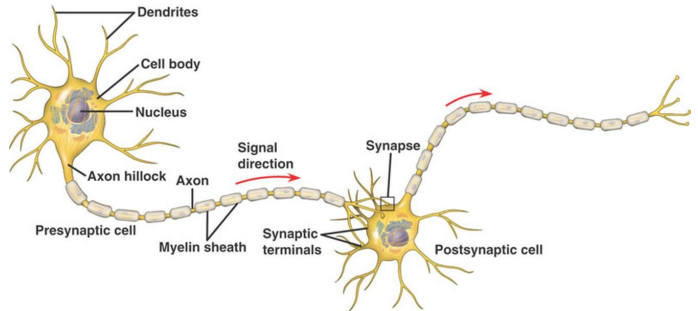
- ▶ **1st bounce:** Direct light from primary rays
→ Antialiasing like for Rasterization
- ▶ **2nd + higher order bounces:** Indirect light from secondary rays
→ Depends on scene visibility

Consequently, efficiently computing this boundary integral
is still ongoing research



Next Lecture: Deep Learning

→ **Deep Learning**



- [1] J. T. Barron. “A General and Adaptive Robust Loss Function”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019.
- [2] S. Laine, J. Hellsten, T. Karras, Y. Seol, J. Lehtinen, and T. Aila. “Modular Primitives for High-Performance Differentiable Rendering”. In: *ACM Transactions on Graphics* (2020).